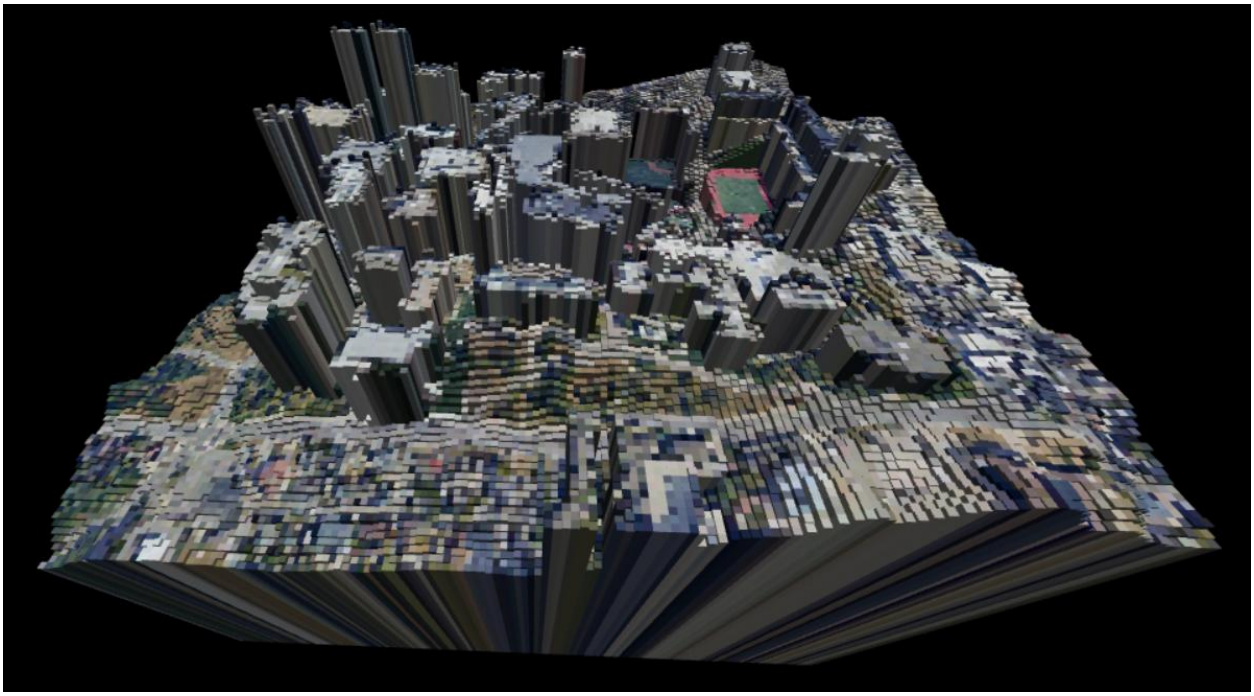
Project 2: Cube World Documentation

Grant Hamilton

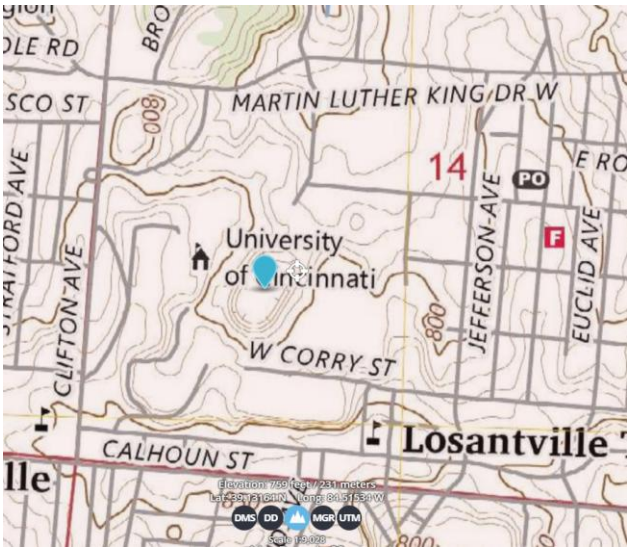


Design Plans, Features, and Implementation

My project demonstrates the rough terrain of the University of Cincinnati's West Campus. I decided I wanted to pursue this early in the timeline, as I've always enjoyed studying maps of familiar places, and I thought it would be funny to exaggerate the elevation features to better reflect how they feel to walk every day…

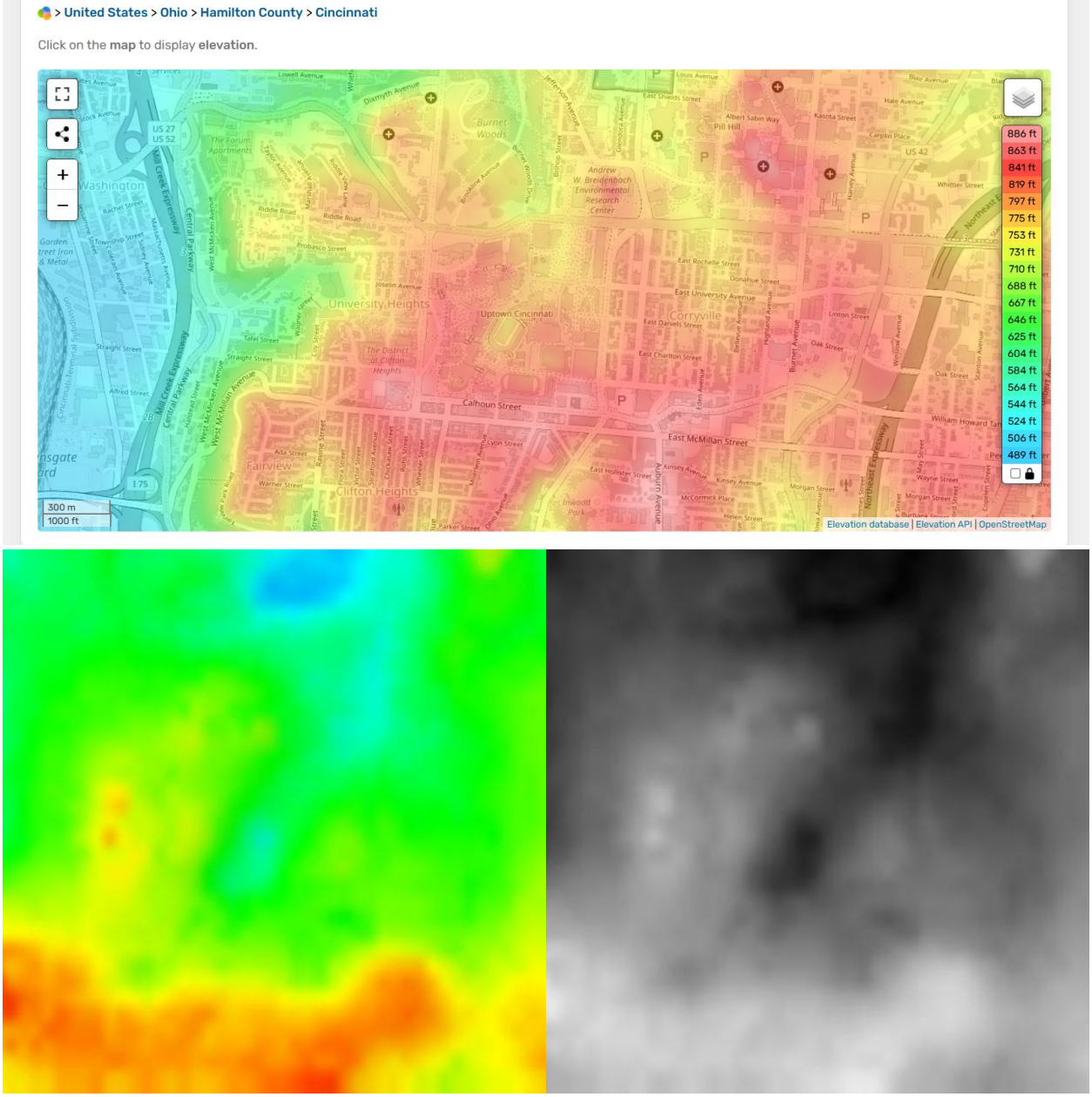
While it isn't too difficult to find the satellite imagery of campus, I wasn't too sure how to find the elevation information. There turned out to be a couple of different avenues for me to go down:

A traditional topographic map would be easy to manually shade in for my script, but I found it a bit annoying that I couldn't find any without text or other extraneous symbols. Along with that, I wanted to find more continuous data.



Topographic Map of UC

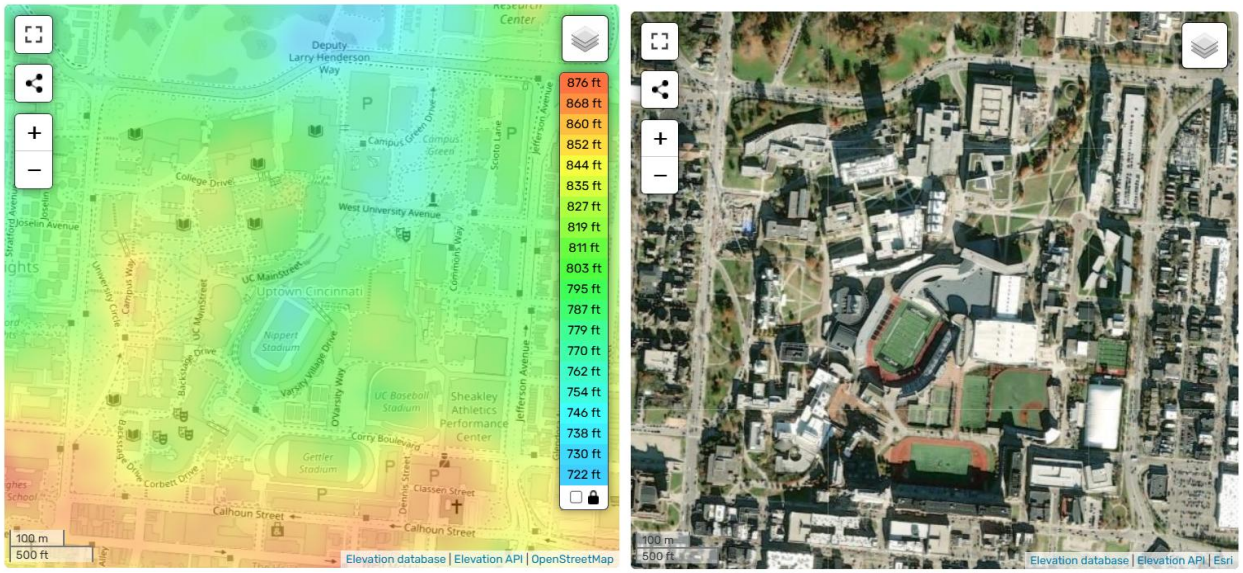
After doing some digging, I found a website with good elevation data. Better yet, I could easily apply some effects to get a grayscale image for the script.



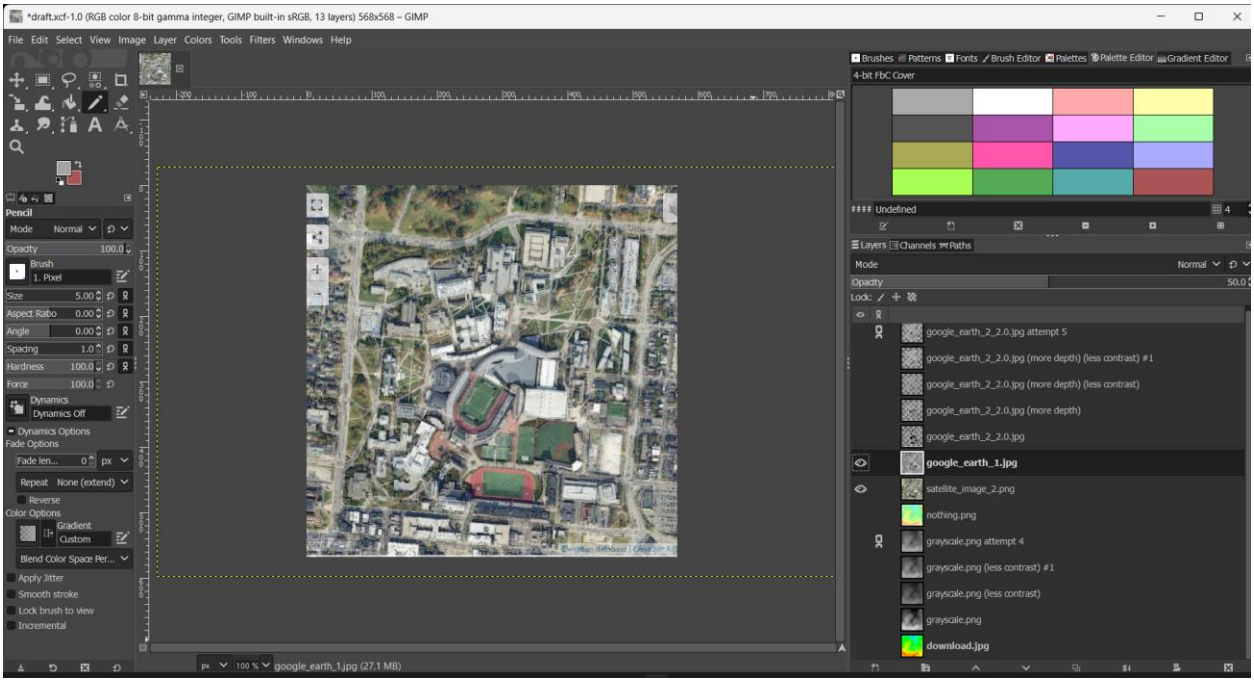
Improved Data of UC Topography

Next, I began work on building my final image for the script:

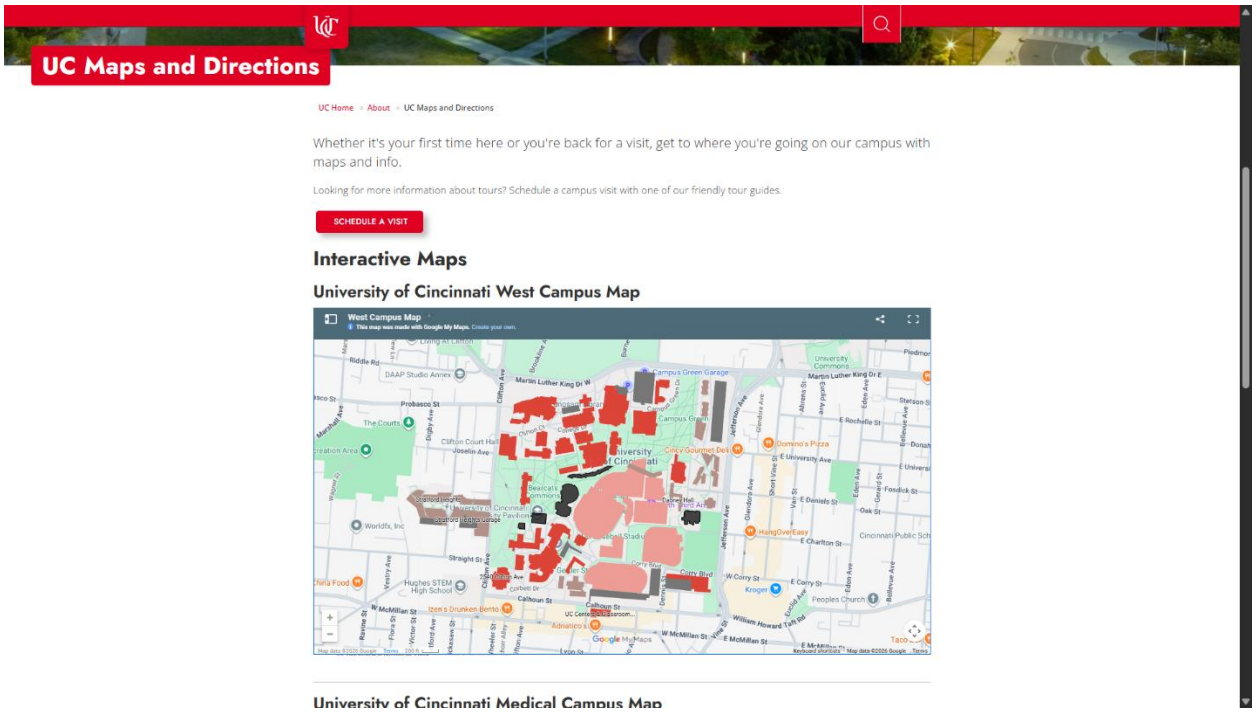
1. I started small by mapping the satellite imagery onto the topography. I used the icons in the top left corner to make sure both images aligned right in GIMP.

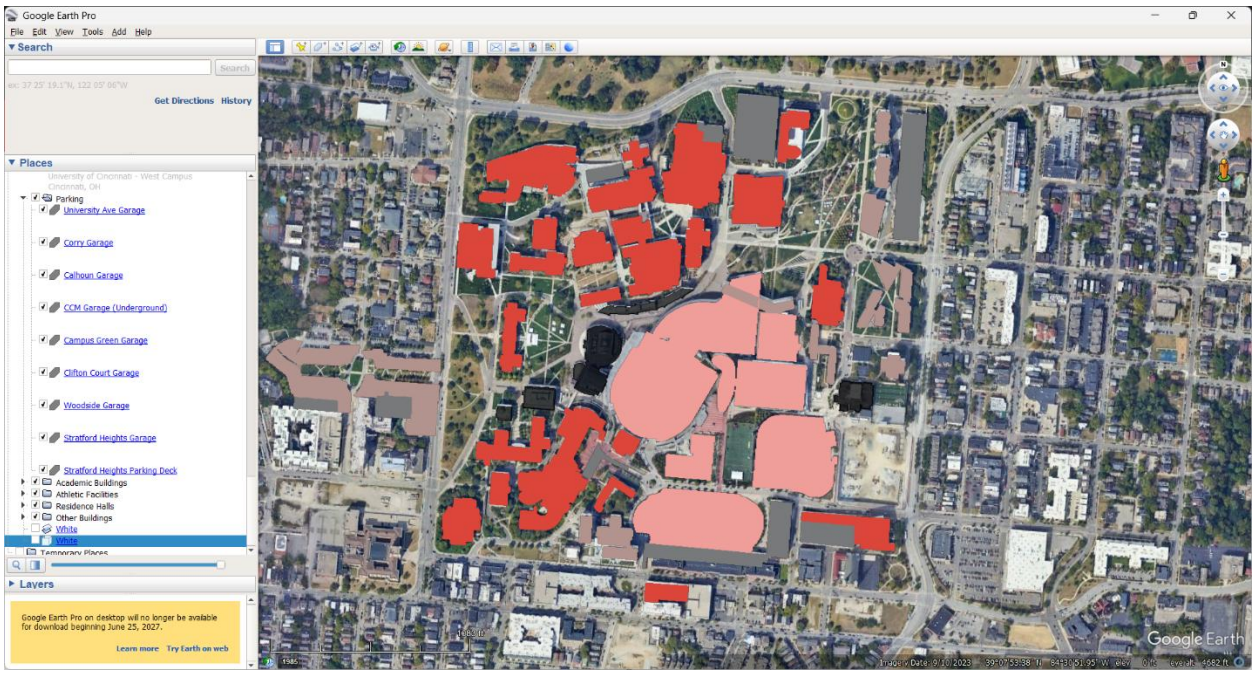


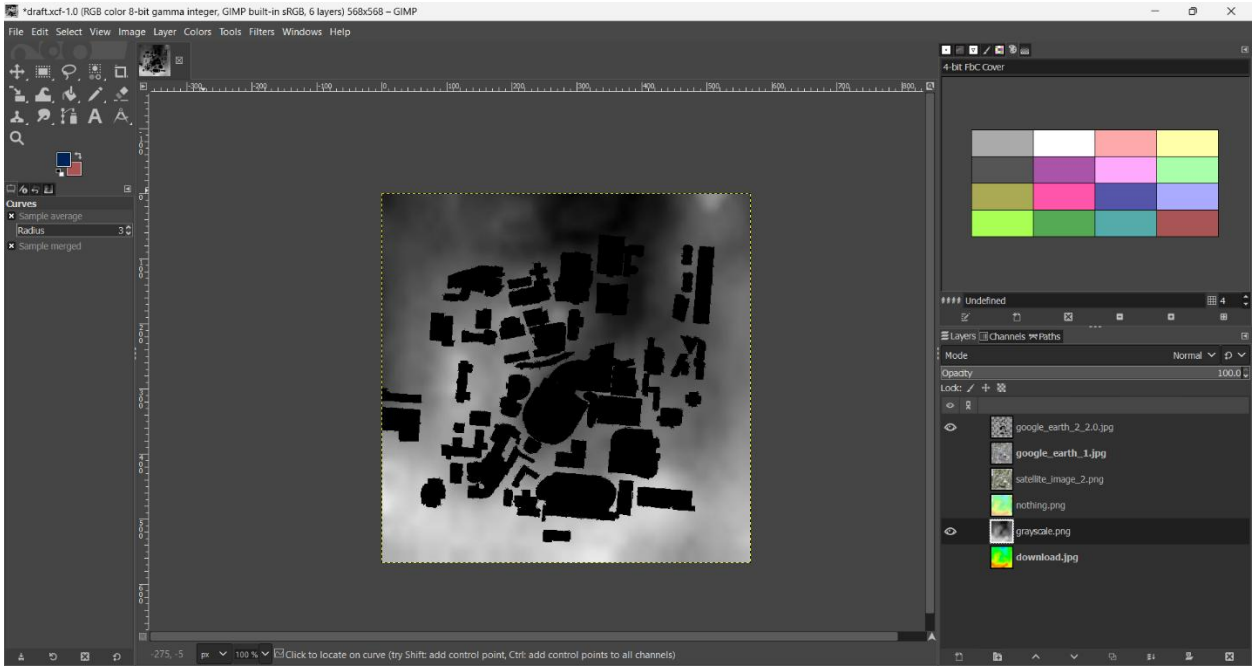
2. Next, I replaced the satellite imagery from the topography website with one from Google Earth. I did this because the original one looked a bit washed out.



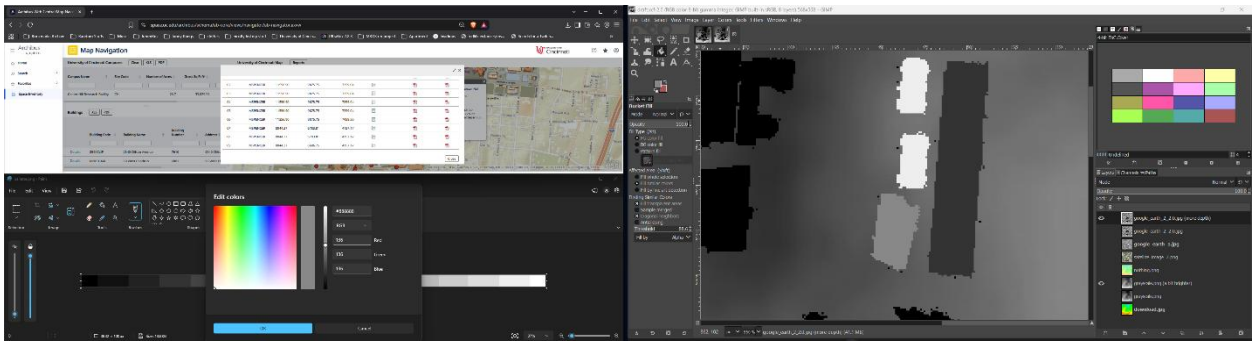
4. Then, I took a .kml file with all the footprints of UC's buildings and placed it on top of my topographic image. This saved me a lot of time from having to draw it out manually.



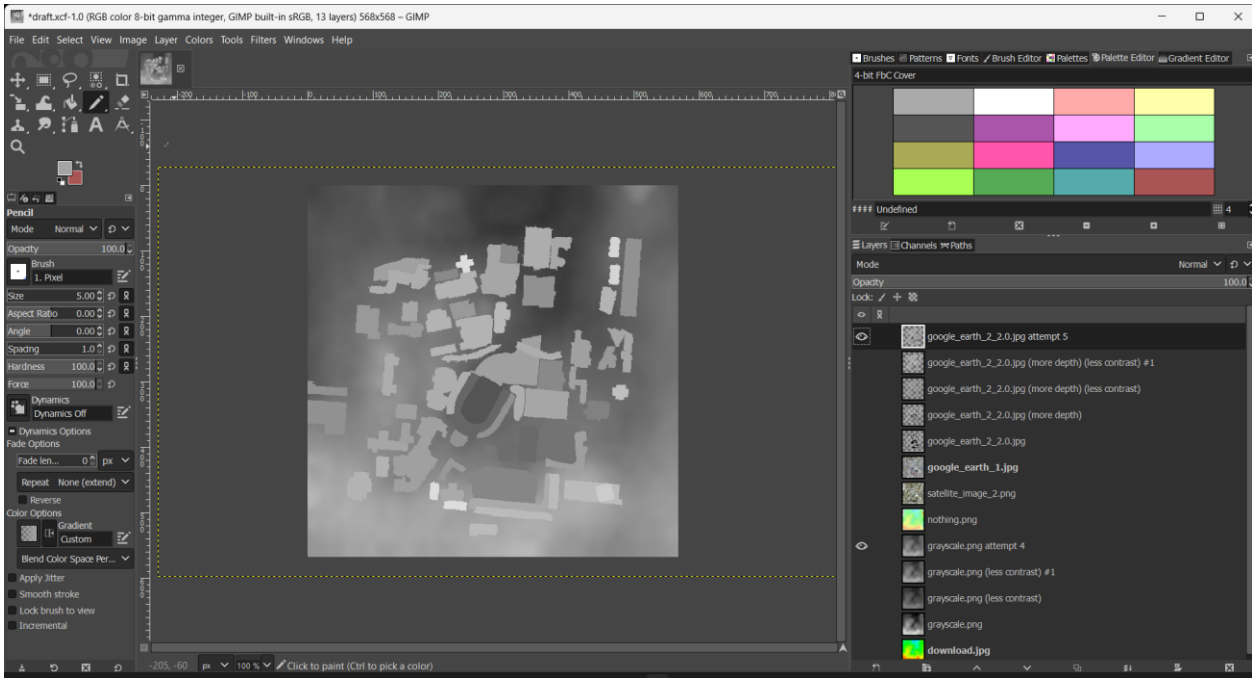


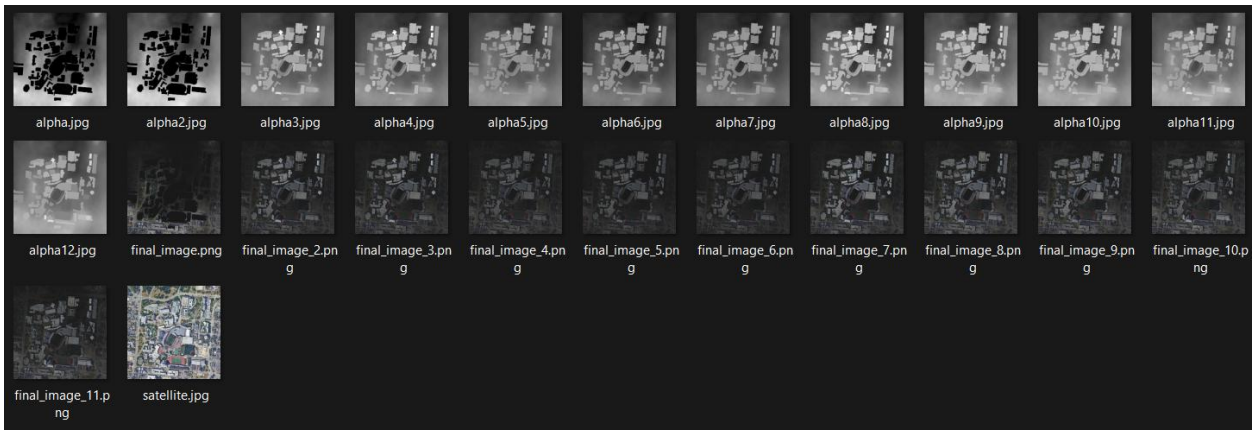


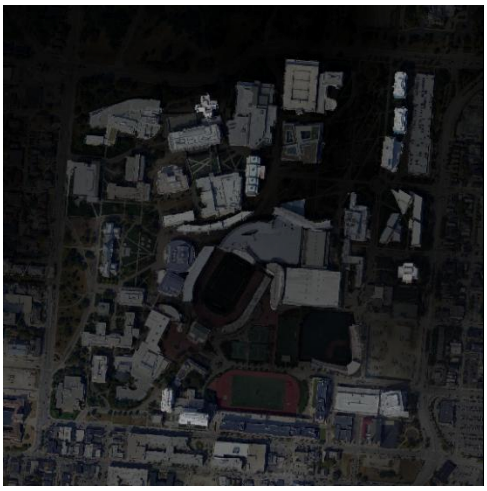
5. Now, I colored all the heights of the buildings and scaled it properly with the topography. I partly went off a website that gave floor numbers for each building, and partly just off memory.



6. I went through many revisions, but I ended up finding a good balance between building height and the general topography:

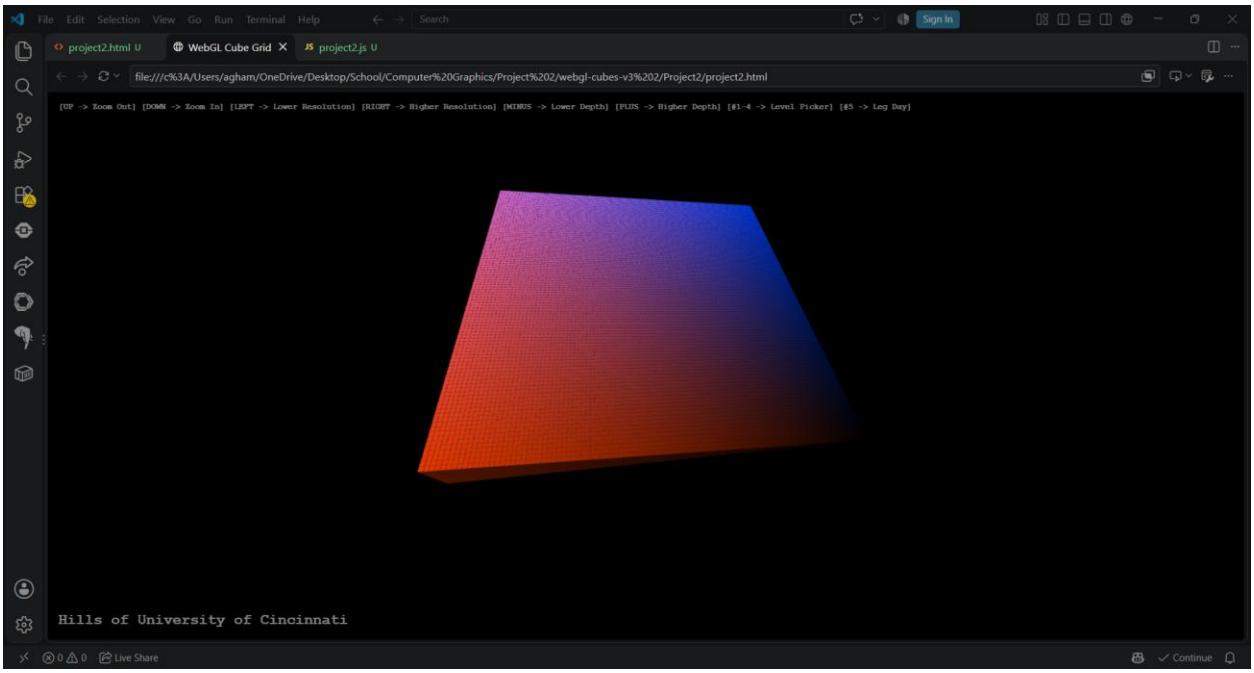


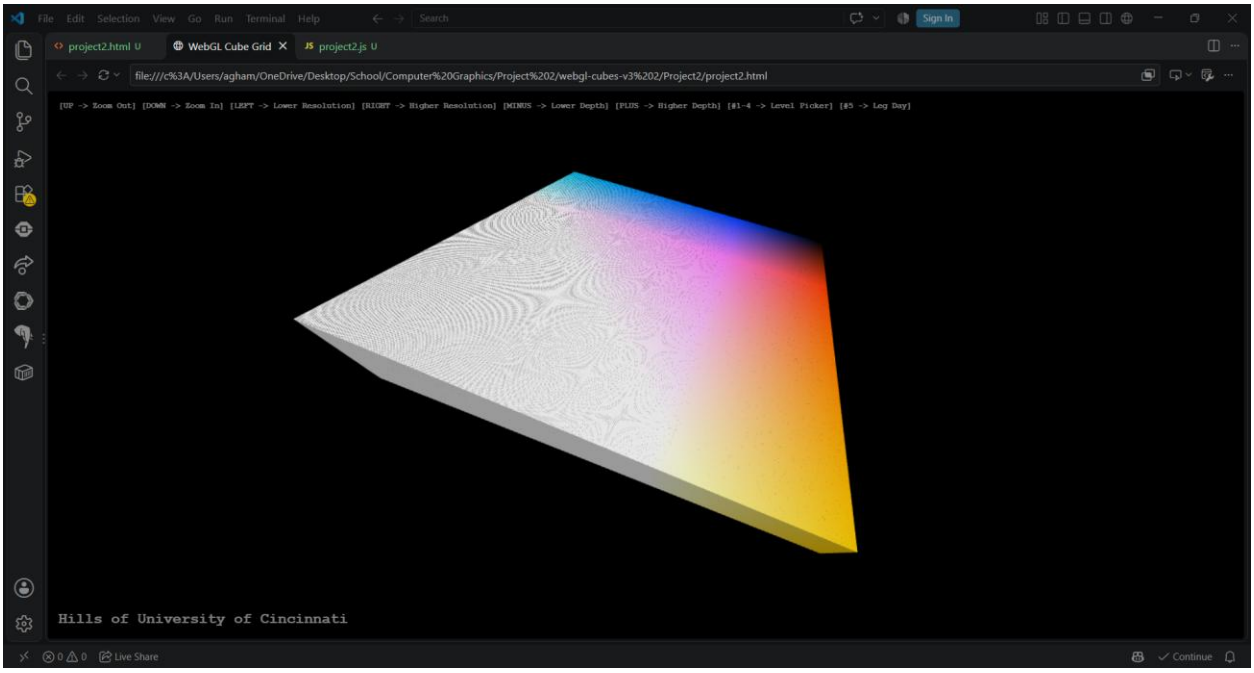




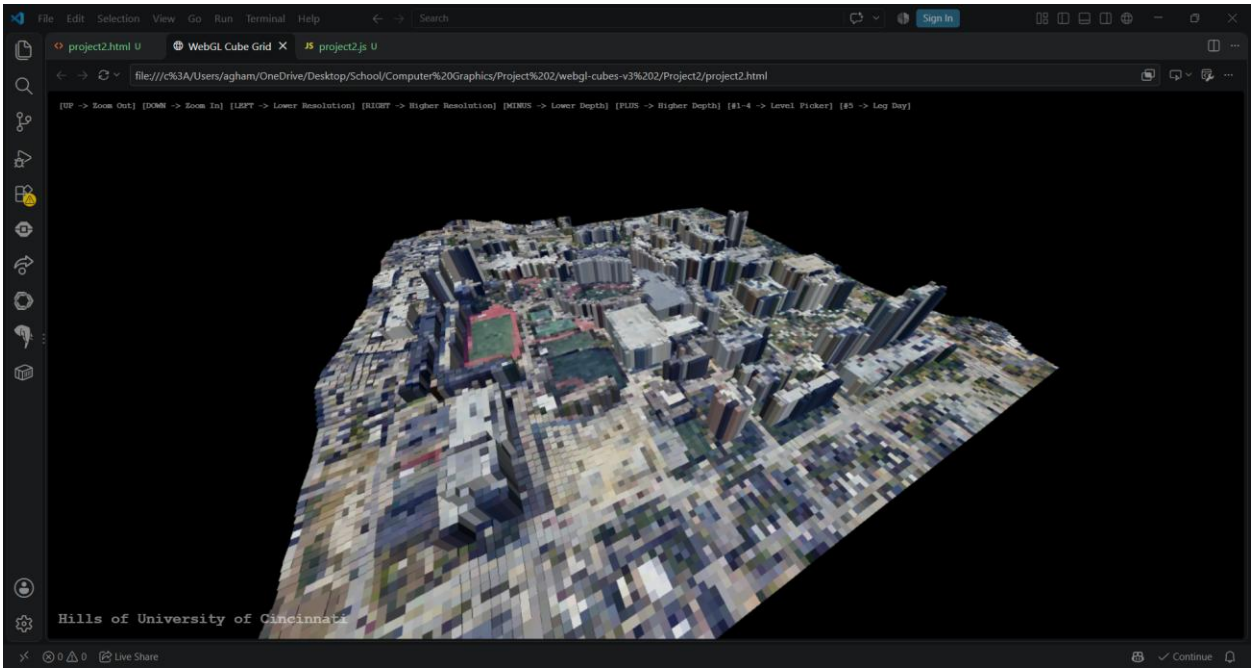
With the final image created, I gave more time to coding.

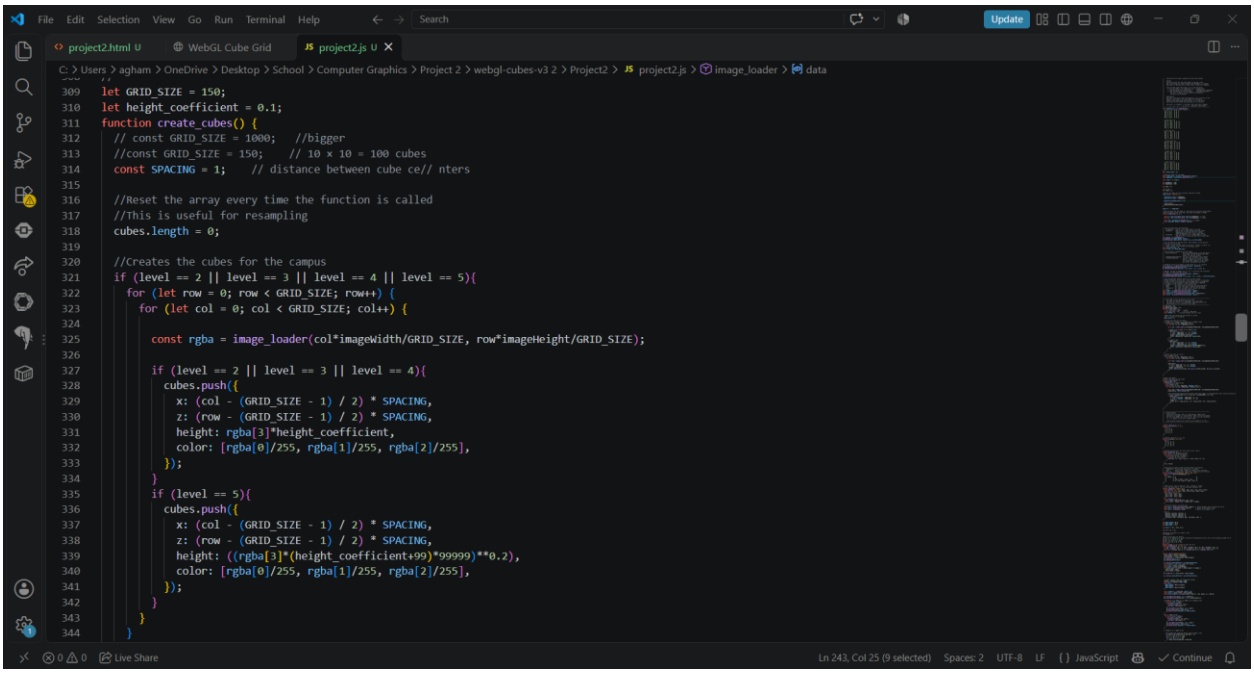
To fulfill parts a-c of level 1, I created a simple procedural landscape which changes height and color based on its row and column. For part d, I implemented the functionality for the camera to rotate around the center.





Level 2 is the default upon running the script. It takes a single image file (rgb for color info, alpha for height info) and applies these values onto a grid of cubes. The user has the option to change the number of cubes (i.e., the resolution) by pressing the left and right arrow keys. To properly sample the image, the image_loader function is given the position of each cube and the pixel that would correspond to it is chosen.





```
let GRID_SIZE = 150;
let height_coefficient = 0.1;
function create_cubes() {
  // const GRID_SIZE = 1000;   //bigger
  //const GRID_SIZE = 150;    // 10 x 10 = 100 cubes
  const SPACING = 1;    // distance between cube ce// nters

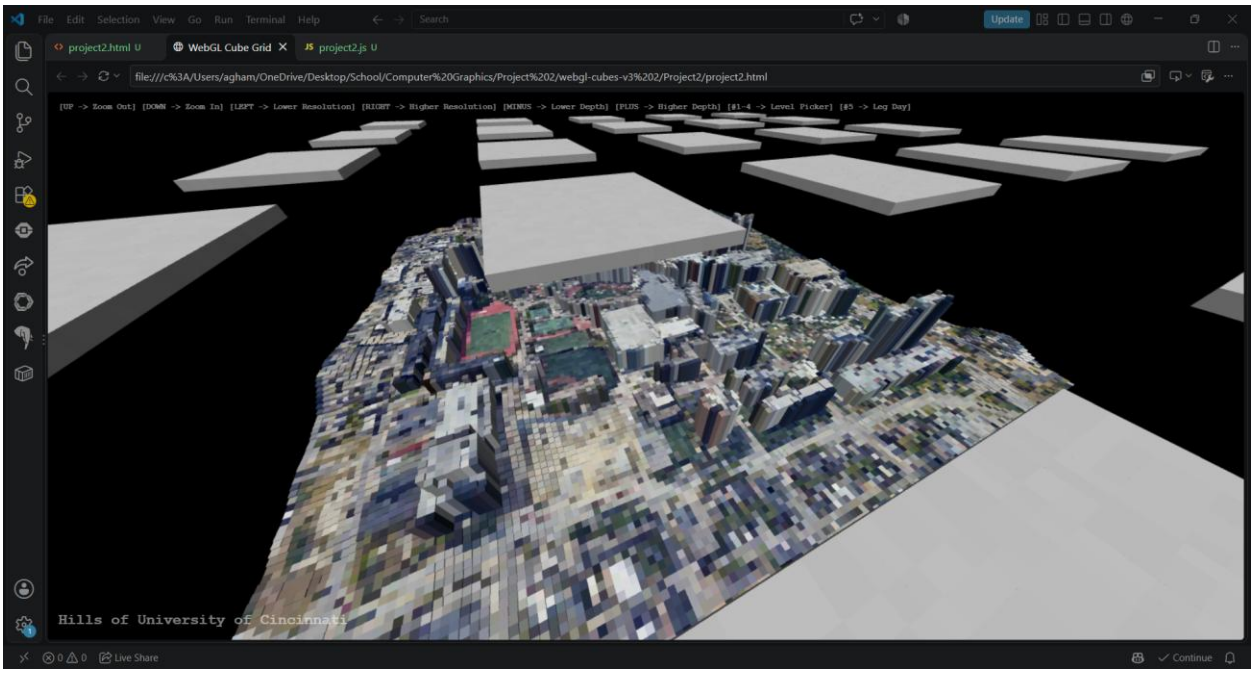
  //Reset the array every time the function is called
  //This is useful for resampling
  cubes.length = 0;

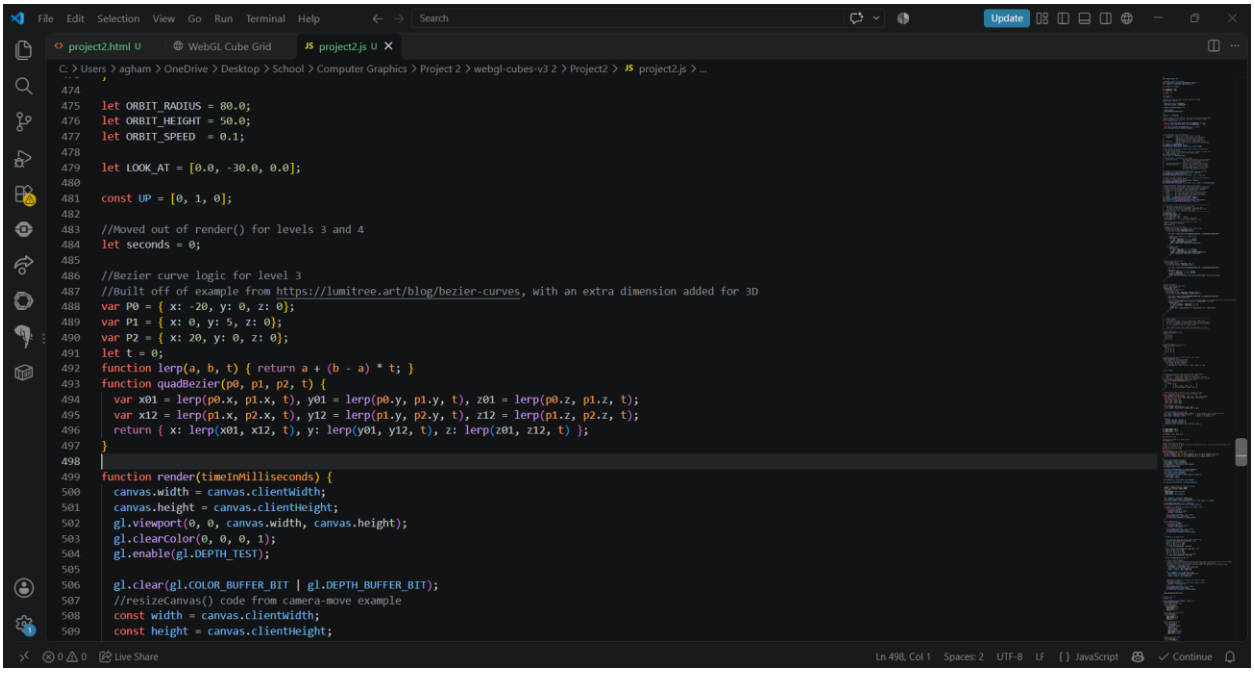
  //Creates the cubes for the campus
  if (level == 2 || level == 3 || level == 4 || level == 5){
    for (let row = 0; row < GRID_SIZE; row++) {
      for (let col = 0; col < GRID_SIZE; col++) {

        const rgba = image_loader(col*imageWidth/GRID_SIZE, row*imageHeight/GRID_SIZE);

        if (level == 2 || level == 3 || level == 4){
          cubes.push({
            x: (col - (GRID_SIZE - 1) / 2) * SPACING,
            z: (row - (GRID_SIZE - 1) / 2) * SPACING,
            height: rgba[3]*height_coefficient,
            color: [rgba[0]/255, rgba[1]/255, rgba[2]/255],
          });
        }
        if (level == 5){
          cubes.push({
            x: (col - (GRID_SIZE - 1) / 2) * SPACING,
            z: (row - (GRID_SIZE - 1) / 2) * SPACING,
            height: ((rgba[3]*(height_coefficient+99)*99999)**0.2),
            color: [rgba[0]/255, rgba[1]/255, rgba[2]/255],
          });
        }
      }
    }
```

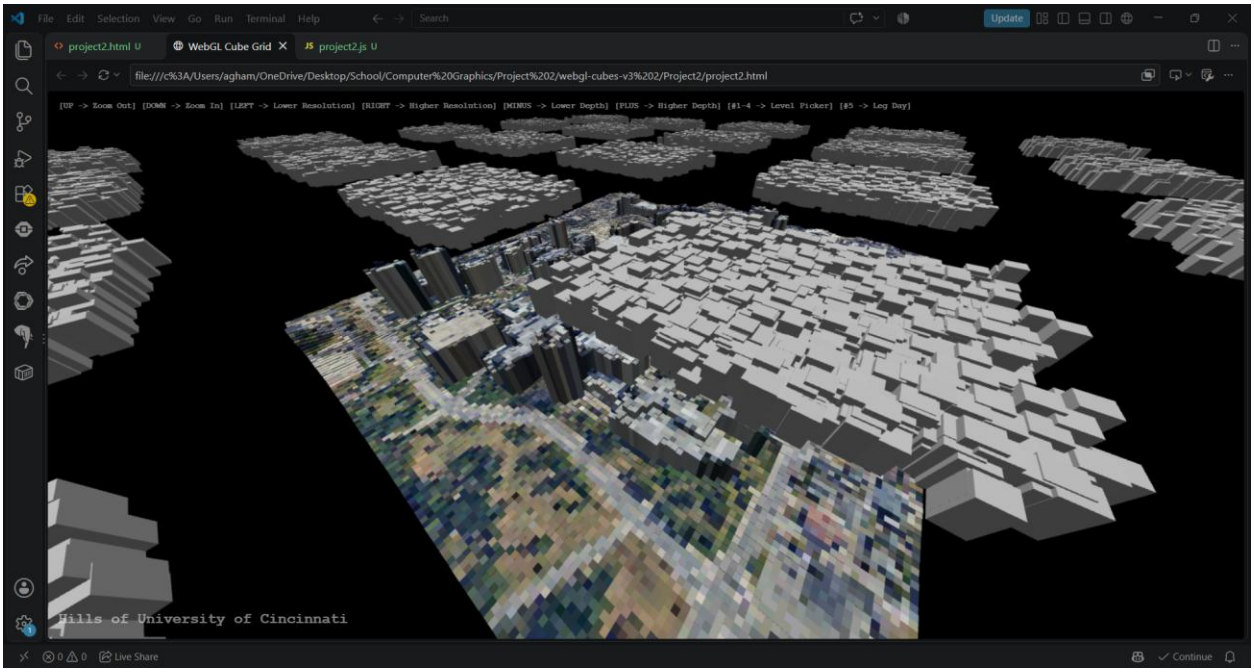
Level 3 adds clouds to the sky, which are translated into place. These clouds are procedurally generated and animate along a path of two quadratic Bezier curves that connect at both ends.

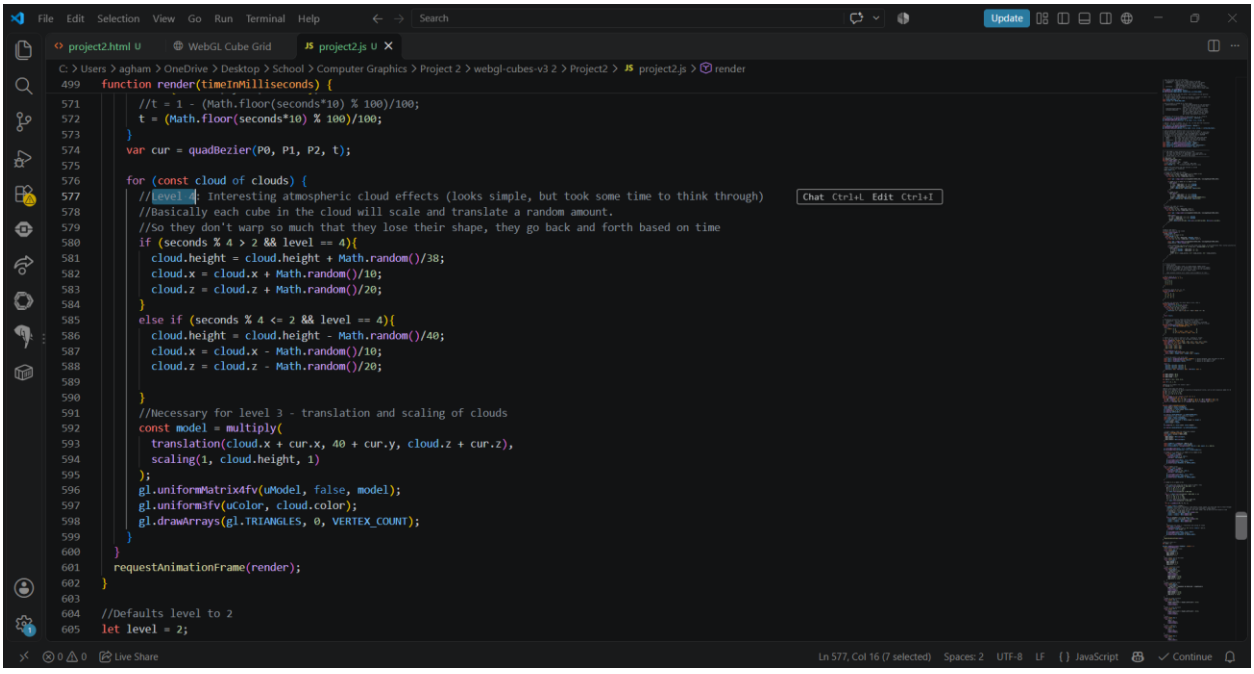




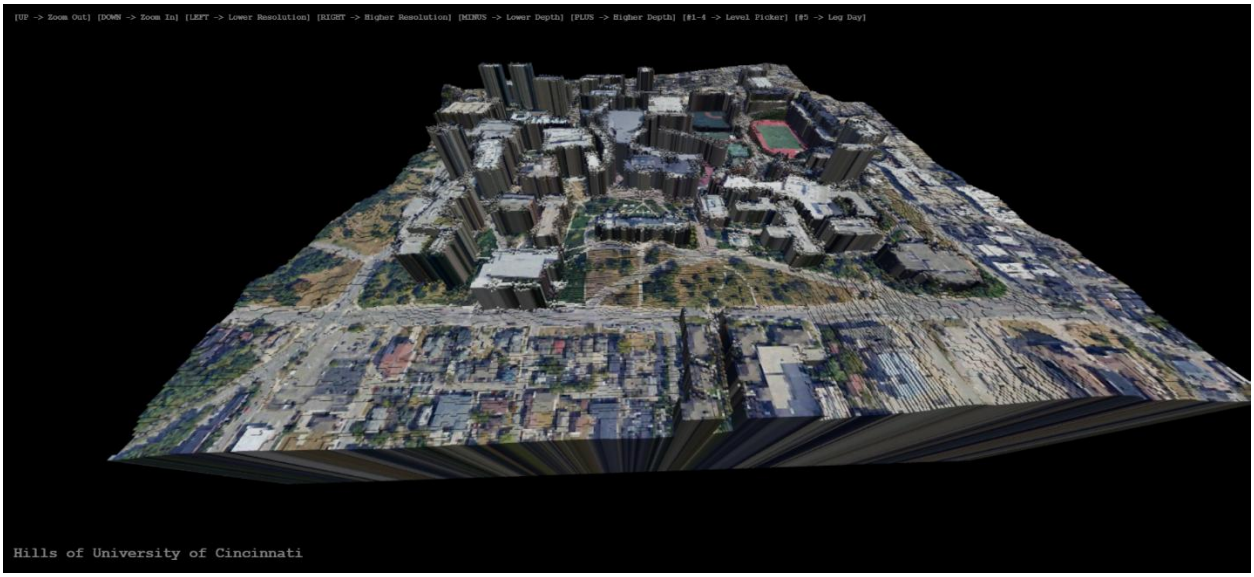
```
474
475  let ORBIT_RADIUS = 80.0;
476  let ORBIT_HEIGHT = 50.0;
477  let ORBIT_SPEED  = 0.1;
478
479  let LOOK_AT = [0.0, -30.0, 0.0];
480
481  const UP = [0, 1, 0];
482
483  //Moved out of render() for levels 3 and 4
484  let seconds = 0;
485
486  //Bezier curve logic for level 3
487  //Built off of example from https://lumitree.art/blog/bezier-curves, with an extra dimension added for 3D
488  var P0 = { x: -20, y: 0, z: 0};
489  var P1 = { x: 0, y: 5, z: 0};
490  var P2 = { x: 20, y: 0, z: 0};
491  let t = 0;
492  function lerp(a, b, t) { return a + (b - a) * t; }
493  function quadBezier(p0, p1, p2, t) {
494    var x01 = lerp(p0.x, p1.x, t), y01 = lerp(p0.y, p1.y, t), z01 = lerp(p0.z, p1.z, t);
495    var x12 = lerp(p1.x, p2.x, t), y12 = lerp(p1.y, p2.y, t), z12 = lerp(p1.z, p2.z, t);
496    return { x: lerp(x01, x12, t), y: lerp(y01, y12, t), z: lerp(z01, z12, t) };
497  }
498
499  function render(timeInMilliseconds) {
500    canvas.width = canvas.clientWidth;
501    canvas.height = canvas.clientHeight;
502    gl.viewport(0, 0, canvas.width, canvas.height);
503    gl.clearColor(0, 0, 0, 1);
504    gl.enable(gl.DEPTH_TEST);
505
506    gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
507    //resizeCanvas() code from camera-move example
508    const width = canvas.clientWidth;
509    const height = canvas.clientHeight;
```

I was having a lot of fun with procedural generation and chose for level 4 to build off this concept. I chose to add atmospheric effects by making the clouds look puffier. To do this, each cube moves in a random direction and scales very slightly, making the collective shape more complex.





At this point I wanted to add a bit extra to the project, so I made a special level 5. Here, the height is scaled logarithmically to make the hills look even steeper. I chose to do this because the script applied more depth to the parts of the map with higher alpha values. For example, the hills are made apparent toward the south of campus, but not in the north where the elevation is lower.



Controls

[UP -> Zoom Out]

[DOWN -> Zoom In]

[LEFT -> Lower Resolution]

[RIGHT -> Higher Resolution]

[MINUS -> Lower Depth]

[PLUS -> Higher Depth]

[#1-4 -> Level Picker]

[#5 -> Leg Day]

Future Work

An interest of mine is reviewing old maps of familiar areas to see what's changed. I think a logical next step for this script could be to automatically combine satellite imagery and topographic data for any location. An interesting use case could be to take a batch of imagery of the same location taken at different times and seeing the change in a 3D perspective. Aerial photography has existed for over a century and can show drastic changes to landscapes – whether that be in a rural area with a new reservoir, or an urban area with buildings coming and going. Applying a height map could be a cool way of visualizing these changes in a more intuitive way.

Links

Source code:

https://github.com/hamil2ag/Computer-Graphics-I/tree/main/Project2

Publicly hosted application:

https://hamil2ag.github.io/Computer-Graphics-I/Project2/project2.html

Video demonstration:

demonstration.mp4